

Proposition d'uniformisation des notations pour l'apprentissage des préférences

Olivier STRAUSS

April 10, 2025

Jour du rapport: Lundi 03 avril, 2025
Destinataires: Agnès RICO
Curtis MARTELET
Christophe MARSALA

1 Objet du rapport

Origine

Ce rapport vise à uniformiser les différentes formules pour l'apprentissage de préférences comme proposé dans l'article [1]. Il fait suite à l'article intitulé *Learning a preference relation with MacSum Choquet Integrals* proposé à la conférence IJCAI 2025.

But

L'objectif est d'apprendre des préférences sur la base de n critères quantitatifs regroupés dans un vecteur $\mathbf{x}$ de n composantes. Le système d'agrégation proposé doit, à partir de deux vecteurs $\mathbf{x}$ et $\mathbf{x}'$, produire deux sorties y et y' dont l'ordre respecte les préférences entre les deux produits, celui associé aux critères $\mathbf{x}$ et celui associé aux critères $\mathbf{x}'$.

Non concerné

Aucune expérimentation n'est proposée dans ce document.

2 Notations

- n est le nombre de critères sur lesquels on calcule le score de préférence.
- $\Omega = \{1, \dots, n\} \subset \mathbb{N}$ est l'ensemble des indices des n critères.
- $\mathbf{x} = [x_1, \dots, x_n] = [x_i]_{i \in \Omega} \in \mathbb{R}^n$ est un vecteur de critères.
- $\boldsymbol{\varphi} = [\varphi_1, \dots, \varphi_n] \subset \mathbb{R}^n$ est le vecteur de paramètres de l'agrégation (celui qui fait l'objet de l'apprentissage).

- $\langle \cdot, \cdot \rangle$ est le produit scalaire de deux vecteurs défini par $\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i \in \Omega} x_i \cdot y_i$.
- Pour tout $A \subseteq \Omega$, $\#(A)$ est le numéro lexicographique de A obtenu à partir de la fonction caractéristique de A :

$$\chi : A \subseteq \Omega \rightarrow \chi(A) \in \{0, 1\}^n,$$

$$\#(A) = \sum_{i \in \Omega} \chi(A)_i \cdot 2^i \in [0, 2^n - 1].$$

par exemple, $n = 5$, l'ensemble A dont la fonction caractéristique donne $\{0, 1, 0, 0, 1\}$ aura un numéro lexicographique $\#(A) = 0 + 2 + 0 + 0 + 2^4 = 18$. $\emptyset$ l'ensemble vide de Ω a pour code lexicographique 0 et Ω a pour code lexicographique $2^n - 1 = 31$. On peut donc associer à chaque sous-ensemble de Ω son code lexicographique, ou bien associer un ensemble A_k à chaque entier $k = 0 \dots (2^n - 1)$.

- un fonction d'ensemble μ est une fonction qui à tout ensemble $A \subseteq \Omega$ associe une valeur réelle :

$$v : A \subseteq \Omega \rightarrow v(A) \in \mathbb{R}.$$

Une fonction d'ensemble quelconque porte aussi le nom de *game*. Elle est telle que $v(\emptyset) = 0$.

Dans ce document, on s'intéresse à trois fonctions d'agrégation non monotones : l'une basée sur un *game* μ , une autre linéaire d'appuyant sur la fonction λ_φ , enfin la troisième s'appuyant sur la fonction médiane de MacSum ν_φ . Dans les trois cas l'agrégation se fait par l'utilisation de l'intégrale de Choquet.

Si on note $\mathcal{A}_v$ l'agrégation associée à la fonction d'ensemble v on a

- $\mathcal{A}_\mu : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \rightarrow \mathcal{A}_\mu(\mathbf{x}) = C_\mu(\mathbf{x})$,
- $\mathcal{A}_{\lambda_\varphi} : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \rightarrow \mathcal{A}_{\lambda_\varphi}(\mathbf{x}) = C_{\lambda_\varphi}(\mathbf{x}) = \sum_{i \in \Omega} x_i \cdot \varphi_i$,
- $\mathcal{A}_{\nu_\varphi} : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \rightarrow \mathcal{A}_{\nu_\varphi}(\mathbf{x}) = \frac{1}{2} \left(C_{\nu_\varphi}(\mathbf{x}) + C_{\nu_\varphi^c}(\mathbf{x}) \right)$.
- $[\cdot]$ est une permutation triant un vecteur $\boldsymbol{\varphi}$ par ordre décroissant : $\varphi_{[1]} \geq \varphi_{[2]} \geq \dots \geq \varphi_{[n]}$, avec $\varphi_{[n+1]} = \varphi_{[n]}$,
- $[\cdot]$ est une permutation triant un vecteur $\boldsymbol{\varphi}$ par ordre croissant : $\varphi_{[1]} \leq \varphi_{[2]} \leq \dots \leq \varphi_{[n]}$ avec $\varphi_{[n+1]} = \varphi_{[n]}$.
- $\forall \mathbf{z} \in \mathbb{R}$, on pose $[z]^+ = \max(0, z)$.
- $\forall \mathbf{z} \in \mathbb{R}$, on pose $sign$ comme étant la fonction associant au vecteur $\mathbf{z}$ le vecteur $\boldsymbol{\xi} = sign(\mathbf{z})$ des signes de ses éléments : $\xi_i = 1$ si $z_i \geq 0$ et $\xi_i = -1$ sinon.

3 Uniformisation des notations

Dans cette uniformisation, chaque agrégation est associée à

- 1) son paramètre $\boldsymbol{\varphi}$,

- 2) une fonction qui associe à chaque vecteur de critères $\mathbf{x}$ un vecteur $\phi(\mathbf{x})$ de même dimension que $\boldsymbol{\varphi}$.

On note $\{A_k\}_{k=0\dots 2^n-1}$ l'ensemble des 2^n sous-ensembles possibles de Ω numérotés lexicographiquement.

Game μ Dans le cas d'un game :

- $\boldsymbol{\varphi} \in \mathbb{R}^{2^n}$ est le vecteur de la transformée de Möbus du game μ . C'est un vecteur de dimension 2^n ,
- le vecteur $\mathbf{X} = \phi(\mathbf{x})$ est obtenu en utilisant les ensembles $\{A_k\}_{k=0\dots 2^n-1}$:

$$X_k = \min_{i \in A_k} x_i.$$

Linéaire λ_φ Dans le cas d'une agrégation linéaire :

- $\boldsymbol{\varphi} \in \mathbb{R}^n$ est le vecteur paramétrique de l'opérateur linéaire. C'est un vecteur de dimension n ,
- le vecteur $\mathbf{X} = \phi(\mathbf{x}) = \mathbf{x}$.

MacSum ν_φ Dans le cas de la médiane de MacSum :

- $\boldsymbol{\varphi} \in \mathbb{R}^n$ est le vecteur paramétrique de l'opérateur MacSum. C'est un vecteur de dimension n ,
- le vecteur $\mathbf{X} = \phi(\mathbf{x})$ est aussi un vecteur de dimension n qui est obtenu de la manière suivante : On pose $\forall k \in [1, n]$, et $[\cdot]$, $[\cdot]$ trient le vecteur $\boldsymbol{\varphi}$,

$$\overline{\alpha}_k^+ = (\max_{i=1}^k x_{[i]} - \max_{i=1}^{k-1} x_{[i]}),$$

$$\overline{\alpha}_k^- = (\min_{i=1}^k x_{[i]} - \min_{i=1}^{k-1} x_{[i]}),$$

$$\underline{\alpha}_k^+ = (\min_{i=1}^k x_{[i]} - \min_{i=1}^{k-1} x_{[i]}),$$

$$\underline{\alpha}_k^- = (\max_{i=1}^k x_{[i]} - \max_{i=1}^{k-1} x_{[i]}),$$

$$\text{avec } \max_{i=1}^0 x_{[i]} = 0 = \min_{i=1}^0 x_{[i]} \text{ et } \min_{i=1}^0 x_{[i]} = 0 = \max_{i=1}^0 x_{[i]}.$$

On note σ la bijection inverse du tri par ordre décroissant : $\forall k \in [1, n]$, $\sigma([k]) = k$. En remarquant que $[k] = [n - k + 1]$ on voit que c'est aussi celle qui permet l'inverse du tri croissant.

On pose alors $\mathbf{X} = \phi(\mathbf{x})$ le vecteur tel que :

$$X_k = \frac{1}{2} \cdot \begin{cases} \overline{\alpha}_{\sigma(k)}^+ + \underline{\alpha}_{\sigma(k)}^-, & \text{si } \varphi_{[k]} \geq 0 \\ \overline{\alpha}_{\sigma(n-k+1)}^- + \underline{\alpha}_{\sigma(n-k+1)}^-, & \text{sinon.} \end{cases}$$

Tout cela étant posé, dans les trois cas l'agrégation $\mathcal{A}$ des valeurs du vecteur $\mathbf{x}$ peut s'écrire comme le produit scalaire entre $\boldsymbol{\varphi}$ le vecteur de paramètres et $\phi(\mathbf{x})$: $\mathcal{A}(\mathbf{x}, \boldsymbol{\varphi}) = \langle \boldsymbol{\varphi}, \phi(\mathbf{x}) \rangle$. Dans le cas de l'opérateur MacSum, le vecteur $\mathbf{X} = \phi(\mathbf{x})$ peut être obtenu par l'utilisation de l'algorithme ??.

4 Apprentissage des préférences

4.1 Les données

Soient deux *produits* dont les critères (ou caractéristiques) sont définis dans les vecteurs $\mathbf{x}$ et $\mathbf{x}'$, on dit que :

- $\mathbf{x} > \mathbf{x}'$ si le produit défini par $\mathbf{x}$ est préféré à celui défini par $\mathbf{x}'$.
- $\mathbf{x} \sim \mathbf{x}'$ si on n'a aucune préférence entre le produit défini par $\mathbf{x}$ et celui défini par $\mathbf{x}'$.

On pose δ le paramètre qui permet de définir, en sortie d'agrégation, la préférence ou la non-préférence. Dans la publication de référence, il est posé $\delta = 0.0001$. soit $y = \mathcal{A}(\mathbf{x}, \boldsymbol{\varphi})$ et $y' = \mathcal{A}(\mathbf{x}', \boldsymbol{\varphi})$, on dira qu'il n'y a pas de préférence entre $\mathbf{x}$ et $\mathbf{x}'$ si $|y - y'| \leq \delta$. Du coup, $\mathbf{x}$ est préféré à $\mathbf{x}'$ se caractérise par $y > y' + \delta$.

Par soucis de simplification, on supposera toujours que $\mathbf{x}$ est préféré (ou indifférent) à $\mathbf{x}'$. Si ce n'est pas le cas, il suffit d'inverser les données de la paire $(\mathbf{x}, \mathbf{x}')$.

4.2 Méthode avec perte quadratique

On note $\mathcal{L}_2(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi})$ la fonction d'erreur faite en utilisant le paramètre $\boldsymbol{\varphi}$ et l'agrégation $\mathcal{A}$ pour modéliser la préférence (ou l'indifférence) entre $\mathbf{x}$ et $\mathbf{x}'$. On pose $y = \langle \boldsymbol{\varphi}, \phi(\mathbf{x}) \rangle$ et $y' = \langle \boldsymbol{\varphi}, \phi(\mathbf{x}') \rangle$.

- si $\mathbf{x} > \mathbf{x}'$, alors $\mathcal{L}_2(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = ([\delta - y + y']^+)^2$,

- si $\mathbf{x} \sim \mathbf{x}'$, alors $\mathcal{L}_2(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = ([|y - y'| - \delta]^+)^2$,

On remarque que, dans le premier cas, si $y > y' + \delta$ alors $\mathcal{L}_2(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = 0$, de même que, dans le second cas, $\mathcal{L}_2(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = 0$ si $|y - y'| < \delta$.

Pour apprendre le paramètre $\boldsymbol{\varphi}$, il faut sélectionner un nombre important de paires $(\mathbf{x}, \mathbf{x}')$ et, pour chaque paire, mettre à jour $\boldsymbol{\varphi}$ par :

- si $\mathbf{x} > \mathbf{x}'$, alors $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta \cdot (\phi(\mathbf{x}') - \phi(\mathbf{x})) [\delta - \Delta]^+$,

- si $\mathbf{x} \sim \mathbf{x}'$, alors $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta \cdot \text{sign}(\Delta) \cdot (\phi(\mathbf{x}) - \phi(\mathbf{x}')) [|\Delta| - \delta]^+$,

avec $\Delta = y - y'$ et β un coefficient d'apprentissage à définir expérimentalement.

4.3 Méthode avec perte absolue

On note $\mathcal{L}_1(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi})$ la fonction d'erreur faite en utilisant le paramètre $\boldsymbol{\varphi}$ et l'agrégation $\mathcal{A}$ pour modéliser la préférence (ou l'indifférence) entre $\mathbf{x}$ et $\mathbf{x}'$. On pose $y = \langle \boldsymbol{\varphi}, \phi(\mathbf{x}) \rangle$ et $y' = \langle \boldsymbol{\varphi}, \phi(\mathbf{x}') \rangle$.

- si $\mathbf{x} > \mathbf{x}'$, alors $\mathcal{L}_1(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = |\delta - y + y'|$,

- si $\mathbf{x} \sim \mathbf{x}'$, alors $\mathcal{L}_1(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = ||y - y'| - \delta|$,

Comme précédemment, dans le premier cas, si $y > y' + \delta$ alors $\mathcal{L}_1(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = 0$, de même que, dans le second cas, $\mathcal{L}_1(\mathbf{x}, \mathbf{x}', \boldsymbol{\varphi}) = 0$ si $|y - y'| < \delta$.

Pour apprendre le paramètre $\boldsymbol{\varphi}$, il faut sélectionner un nombre important de paires $(\mathbf{x}, \mathbf{x}')$ et, pour chaque paire, mettre à jour $\boldsymbol{\varphi}$ au tour t par :

- si $\mathbf{x} > \mathbf{x}'$, alors on pose $\Gamma = (\phi(\mathbf{x}') - \phi(\mathbf{x}))$ si $\Delta < \delta$ et $\Gamma = 0$ sinon,

- si $\mathbf{x} \sim \mathbf{x}'$, alors on pose $\Gamma = 0$ si $|\Delta| < \delta$ et $\Gamma = \text{sign}(\Delta) (\phi(\mathbf{x}) - \phi(\mathbf{x}'))$ sinon.

avec $\Delta = y - y'$, $\mathbf{0}$ le vecteur nul de même dimension que $\boldsymbol{\varphi}$.

Ensuite on pose $\bar{\Gamma} \leftarrow \bar{\Gamma} + \frac{1}{t} (\Gamma - \bar{\Gamma})$, puis on met à jour $\boldsymbol{\varphi}$ par :

$\boldsymbol{\varphi} \leftarrow -\text{sign}(\bar{\Gamma}) \cdot \frac{\sqrt{\bar{\Gamma}}}{\gamma} [|\bar{\Gamma}| - \beta]^+$, où β et γ sont des coefficients à définir (dans la publication de référence, $\beta = 0.1$ et $\gamma = 1000$) et $\text{sign}(\bar{\Gamma})$

5 Utilisation d'une base de données de régression pour simuler l'apprentissage

Comme on ne dispose généralement pas d'une base de donnée de préférences, on propose de simuler ces données en utilisant une base de régression. Une base de régression est un ensemble de m paires $(\mathbf{x}_i, z_i)$, $i = 1 \dots m$ où $\mathbf{x}_i \in \mathbb{R}^n$ et $z_i \in \mathbb{R}$.

Pour simuler une préférence, il faut définir un paramètre ϵ (différent de δ) permettant de simuler l'indifférence. Supposons qu'on veuille faire un apprentissage sur $T = 4000$ itérations.

Pour chaque itération $t = 1 \dots T$

tirer au hasard $i, j \in [1, m]$

– si $z_i > z_j$ poser $\mathbf{x} = \mathbf{x}_i$ et $\mathbf{x}' = \mathbf{x}_j$ sinon poser $\mathbf{x}' = \mathbf{x}_i$ et $\mathbf{x} = \mathbf{x}_j$.

– si $|z_i - z_j| < \epsilon$ poser $\mathbf{x} \sim \mathbf{x}'$ sinon poser $\mathbf{x} > \mathbf{x}'$.

– faire la mise à jour du paramètre.

6 Cas MacSum sans valeur de tolérance δ

Un des avantages potentiel de MacSum est son aptitude à générer des données imprécises, ce qui permet éventuellement de ne pas avoir à fixer une valeur de tolérance. Donc cette fois on considère l'algorithme de calcul de MacSum sous son aspect intervalliste : $\mathcal{A}_{\nu_\varphi} : \mathbb{R}^n \rightarrow \mathbb{IR}$, $\mathbf{x} \rightarrow \mathcal{A}_{\nu_\varphi}(\mathbf{x}) = [C_{\nu_\varphi}(\mathbf{x}), C_{\nu_\varphi}(\mathbf{x})]$. Pour une paire $(\mathbf{x}, \mathbf{x}')$ telle que soit $\mathbf{x} \sim \mathbf{x}'$ soit $\mathbf{x} > \mathbf{x}'$, l'agrégation MacSum produit deux intervalles : $[\underline{y}, \bar{y}] = A_{\nu_\varphi}(\mathbf{x})$ et $[\underline{y}', \bar{y}'] = A_{\nu_\varphi}(\mathbf{x}')$.

On peut alors considérer deux types d'ordre :

- un ordre basé sur les treillis, ou ordre faible :
on dit que $\mathbf{x} > \mathbf{x}'$ si $\underline{y} > \underline{y}'$ et $\bar{y} > \bar{y}'$,
on dit que $\mathbf{x} \sim \mathbf{x}'$ si $\underline{y} > \underline{y}'$ et $\bar{y} \leq \bar{y}'$ ou le contraire ; autrement dit un des intervalles est contenu dans l'autre. Cette situation peut aisément être détectée en regardant si la variable $(\underline{y} - \underline{y}') \cdot (\bar{y} - \bar{y}')$ est négative.
- un ordre basé sur la dominance ou ordre fort :
on dit que $\mathbf{x} > \mathbf{x}'$ si $\underline{y} > \bar{y}'$,
on dit que $\mathbf{x} \sim \mathbf{x}'$ si $\bar{y} \geq \underline{y}'$ ou $\bar{y}' \geq \underline{y}$, autrement dit les intervalles s'intersectent. Cette situation peut aisément être détectée en regardant si la variable $(\bar{y} - \underline{y}') \cdot (\underline{y} - \bar{y}')$ est négative.

On définit deux vecteurs (mêmes notation que la version mediane) :

$[\underline{X}, \bar{X}] = [\underline{\phi}(x), \bar{\phi}(x)]$ le vecteur tel que :

$$\underline{X}_k = \begin{cases} \underline{\alpha}_{\sigma(k)}^+, & \text{si } \varphi_{[k]} \geq 0 \\ \underline{\alpha}_{\sigma(n-k+1)}^-, & \text{sinon} \end{cases} \quad \bar{X}_k = \begin{cases} \bar{\alpha}_{\sigma(k)}^+, & \text{si } \varphi_{[k]} \geq 0 \\ \bar{\alpha}_{\sigma(n-k+1)}^-, & \text{sinon} \end{cases}$$

6.1 Méthode avec perte quadratique

6.1.1 Ordre faible

On note $\mathcal{L}_2(x, x', \varphi)$ la fonction d'erreur faite en utilisant le paramètre φ et l'agrégation $\mathcal{A}$ pour modéliser la préférence (ou l'indifférence) entre x et x' . $[\underline{y}, \bar{y}] = A_{\nu_\varphi}(x)$ et $[\underline{y}', \bar{y}'] = A_{\nu_\varphi}(x')$.

- si $x \sim x'$, alors $\mathcal{L}_2(x, x', \varphi) = \begin{cases} 0, & \text{si } (\underline{y} - \underline{y}') \cdot (\bar{y} - \bar{y}') \leq 0 \\ \max\left((\underline{y} - \underline{y}'), (\bar{y} - \bar{y}')\right)^2, & \text{sinon} . \end{cases}$
- si $x > x'$, alors $\mathcal{L}_2(x, x', \varphi) = ([\underline{y}' - \underline{y}]^+)^2 + ([\bar{y}' - \bar{y}]^+)^2$.

Pour apprendre le paramètre φ , il faut sélectionner un nombre important de paires (x, x') et, pour chaque paire, mettre à jour φ par :

- si $x \sim x'$, alors si $(\bar{y} - \bar{y}') \cdot (\underline{y} - \underline{y}') \leq 0$

$$\varphi \leftarrow \varphi - \beta \cdot \begin{cases} \left((\underline{\phi}(x) - \underline{\phi}(x')) \cdot (\underline{y} - \underline{y}') \right), & \text{si } |\bar{y} - \bar{y}'| \leq |\underline{y} - \underline{y}'| \\ \left((\bar{\phi}(x) - \bar{\phi}(x')) \cdot (\bar{y} - \bar{y}') \right), & \text{sinon} . \end{cases}$$

- si $x > x'$, alors $\varphi \leftarrow \varphi - \beta \cdot \left((\underline{\phi}(x') - \underline{\phi}(x)) [\underline{y}' - \underline{y}]^+ + (\bar{\phi}(x') - \bar{\phi}(x)) [\bar{y}' - \bar{y}]^+ \right)$,

avec β un coefficient d'apprentissage à définir expérimentalement.

6.1.2 Ordre fort

On note $\mathcal{L}_2(x, x', \varphi)$ la fonction d'erreur faite en utilisant le paramètre φ et l'agrégation $\mathcal{A}$ pour modéliser la préférence (ou l'indifférence) entre x et x' . $[\underline{y}, \bar{y}] = A_{\nu_\varphi}(x)$ et $[\underline{y}', \bar{y}'] = A_{\nu_\varphi}(x')$.

- si $x \sim x'$, alors $\mathcal{L}_2(x, x', \varphi) = \begin{cases} 0, & \text{si } (\underline{y} - \bar{y}') \cdot (\bar{y} - \underline{y}') \leq 0 \\ \max\left((\underline{y} - \bar{y}')^2, (\bar{y} - \underline{y}')^2\right), & \text{sinon} . \end{cases}$
- si $x > x'$, alors $\mathcal{L}_2(x, x', \varphi) = ([\bar{y}' - \underline{y}]^+)^2$.

Pour apprendre le paramètre φ , il faut sélectionner un nombre important de paires (x, x') et, pour chaque paire, mettre à jour φ par :

- si $\mathbf{x} \sim \mathbf{x}'$, alors si $(\bar{y} - \underline{y}').(\underline{y} - \bar{y}') \leq 0$

$$\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta \cdot \begin{cases} \left((\bar{\phi}(\mathbf{x}) - \underline{\phi}(\mathbf{x}')).(\bar{y} - \underline{y}') \right), & \text{si } |\bar{y} - \underline{y}'| \leq |\underline{y} - \bar{y}'| \\ \left((\underline{\phi}(\mathbf{x}) - \bar{\phi}(\mathbf{x}')).(\underline{y} - \bar{y}') \right), & \text{sinon} . \end{cases}$$
 - si $\mathbf{x} > \mathbf{x}'$, alors $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta \cdot \left(\bar{\phi}(\mathbf{x}') - \underline{\phi}(\mathbf{x}) \right) [\bar{y}' - \underline{y}]^+$,
- avec β un coefficient d'apprentissage à définir expérimentalement.

7 Calcul de MacSum

Comme pour les autres méthodes, le calcul de l'opérateur MacSum nécessite le calcul des vecteurs $\phi(\mathbf{x})$ et $\bar{\phi}(\mathbf{x})$. Ce calcul est effectué de façon récursive. Il nécessite l'usage d'un algorithme de tri indexé. Pour simplifier les notation, posons $\underline{\Phi} = \underline{\phi}(\mathbf{x})$ et $\bar{\Phi} = \bar{\phi}(\mathbf{x})$. L'algorithme s'initialise par $\underline{\Phi} = [0, \dots, 0]$ et $\bar{\Phi} = [0, \dots, 0]$. L'algorithme de tri indexé s'écrit $(\mathbf{x}^t, \iota) = \text{Tri}(\mathbf{x}, \text{sens})$ où sens est le sens du tri (+ croissant, - décroissant), $\mathbf{x}^t$ est le vecteur $\mathbf{x}$ trié et ι le vecteur des indices de tri tel que $\forall k = 1 \dots n \ x_k^t = x_{\iota_k}$.

8 Conclusions

A voir....

[1] Margot Herin, Patrice Perny, Natalia Sokolovska, Learning preference models with sparse interactions of criteria, IJCAI 2023, Pages 3786–3794,

Algorithm 1: Computation of $\underline{X} = \underline{\phi}(x)$, $\bar{X} = \bar{\phi}(x)$

Input: $x = \{x_i\}_{i=1\dots n}$, $\varphi = \{\varphi_i\}_{i=1\dots n}$

Output: $\underline{X}$, $\bar{X}$

set $\iota = \{1, \dots, N\}$;

sort (φ, ι) w.r.t. φ in increasing order ;

$\underline{x} = x_{\iota_1}$, $\bar{x} = x_{\iota_1}$;

if $\varphi_1 \leq 0$ **then**

$\bar{X}_{\iota_1} = \underline{x}$;

$\underline{X}_{\iota_1} = \bar{x}$;

for $k = 2 \dots n$ **do**

if $\varphi_k \leq 0$ **then**

$\bar{X}_{\iota_k} = \bar{X}_{\iota_k} - \underline{x}$;

$\underline{X}_{\iota_k} = \underline{X}_{\iota_k} - \bar{x}$;

$\underline{x} = \min(\underline{x}, x_{\iota_k})$;

$\bar{x} = \max(\bar{x}, x_{\iota_k})$;

if $\varphi_k \leq 0$ **then**

$\bar{X}_{\iota_k} = \bar{X}_{\iota_k} + \underline{x}$;

$\underline{X}_{\iota_k} = \underline{X}_{\iota_k} + \bar{x}$;

sort (φ, ι) w.r.t. φ in decreasing order (or reverse the order of the elements) ;

$\underline{x} = x_{\iota_1}$, $\bar{x} = x_{\iota_1}$;

if $\varphi_1 \geq 0$ **then**

$\bar{X}_{\iota_1} = \bar{X}_{\iota_1} + \bar{x}$;

$\underline{X}_{\iota_1} = \underline{X}_{\iota_1} + \underline{x}$;

for $k = 2 \dots n$ **do**

if $\varphi_k \geq 0$ **then**

$\bar{X}_{\iota_k} = \bar{X}_{\iota_k} - \bar{x}$;

$\underline{X}_{\iota_k} = \underline{X}_{\iota_k} - \underline{x}$;

$\underline{x} = \min(\underline{x}, x_{\iota_k})$;

$\bar{x} = \max(\bar{x}, x_{\iota_k})$;

if $\varphi_k \geq 0$ **then**

$\bar{X}_{\iota_k} = \bar{X}_{\iota_k} + \bar{x}$;

$\underline{X}_{\iota_k} = \underline{X}_{\iota_k} + \underline{x}$;

Algorithm 2: Computation of $X = \phi(x)$

Input: $x = \{x_i\}_{i=1\dots n}$, $\varphi = \{\varphi_i\}_{i=1\dots n}$

Output: X

set $\iota = \{1, \dots, N\}$;

set $X = \mathbf{0} = \{0, \dots, 0\}$;

sort (φ, ι) w.r.t. φ in increasing order ;

$\underline{x} = x_{\iota_1}$, $\bar{x} = x_{\iota_1}$;

if $\varphi_1 \leq 0$ **then**

└ $X_{\iota_1} = \underline{x} + \bar{x}$;

for $k = 2 \dots n$ **do**

└ **if** $\varphi_k \leq 0$ **then**

└ $X_{\iota_k} = X_{\iota_k} - \underline{x} - \bar{x}$;

└ $\underline{x} = \min(\underline{x}, x_{\iota_k})$;

└ $\bar{x} = \max(\bar{x}, x_{\iota_k})$;

└ **if** $\varphi_k \leq 0$ **then**

└ $X_{\iota_k} = X_{\iota_k} + \underline{x} + \bar{x}$;

sort (φ, ι) w.r.t. φ in decreasing order (or reverse the order of the elements) ;

$\underline{x} = x_{\iota_1}$, $\bar{x} = x_{\iota_1}$;

if $\varphi_1 \geq 0$ **then**

└ $X_{\iota_1} = X_{\iota_1} + \bar{x} + \underline{x}$;

for $k = 2 \dots n$ **do**

└ **if** $\varphi_k \geq 0$ **then**

└ $X_{\iota_k} = X_{\iota_k} - \bar{x} - \underline{x}$;

└ $\underline{x} = \min(\underline{x}, x_{\iota_k})$;

└ $\bar{x} = \max(\bar{x}, x_{\iota_k})$;

└ **if** $\varphi_k \geq 0$ **then**

└ $X_{\iota_k} = X_{\iota_k} + \bar{x} + \underline{x}$;

for $k = 1 \dots n$ **do**

└ $X_k = \frac{1}{2} X_k$;
